DEVOPS ENGINEER

Interview Field Guide

Linux • Bash • Python • Docker • Kubernetes • AWS • CI/CD

A practical, command-first reference for diagnosing incidents, writing small automation scripts, and explaining your reasoning during a fresher DevOps interview.

PRIMARY GOAL	Use the same pattern for every scenario: establish the symptom, gather evidence, form a hypothesis, make the smallest safe change, verify recovery, and document the result.

How to use this guide

- Start with the quick triage checklist and Linux command map. Find the exact symptom before jumping to a fix.
- Copy one code block at a time. Replace placeholders such as SERVICE_NAME, POD_NAME, REGION, and INSTANCE_ID.
- Prefer read-only checks first. Restarting, deleting, scaling, changing permissions, or deploying are state-changing actions.
- Use scripts on a test container or non-production host first. Many examples deliberately report findings instead of automatically killing processes or deleting files.

Repository fit

Your existing Notes-Devops_For_Lazy_People repository already has Git/GitHub, Docker, GitHub Actions, Terraform, Ansible, and introductory AWS notes. This companion guide concentrates on operational debugging and beginner-friendly Bash/Python automation; it is not a replacement for your CI/CD workflow notes.

Existing notes: https://github.com/codewithmonil/Notes-Devops_For_Lazy_People

Contents

1. 1. Incident response: a repeatable troubleshooting method
2. 2. Linux command map and high CPU, memory, disk, process, service, and network diagnosis
3. 3. Bash scripting essentials and ready-to-run operations scripts
4. 4. Python scripting essentials and safe monitoring / log parsing scripts
5. 5. Docker troubleshooting runbook
6. 6. Kubernetes troubleshooting runbook
7. 7. AWS / EC2 / CloudWatch / IAM checks
8. 8. CI/CD deployment failure runbook
9. 9. High-availability and performance scenarios
10. 10. Mock interview scenarios and answer frameworks
11. 11. Quick command index and official references

1. Incident response: the repeatable method

The 7-step troubleshooting loop

12. Clarify impact: one user or all users? one pod or all replicas? started after a deployment or gradually?
13. Check scope: host, process, container, Kubernetes workload, cloud resource, or external dependency.
14. Measure current state: CPU, memory, disk, network, restarts, error rate, latency, and recent events.
15. Read evidence: service logs, application logs, kernel logs, CI/CD logs, and deployment events. Use a relevant time window.
16. Form a testable hypothesis: for example, “disk is full because old logs are growing”, not “Linux is broken”.
17. Make the smallest reversible fix; preserve logs and avoid killing/restarting resources before understanding impact.
18. Verify: health endpoint, error rate, process/service status, rollout status, and user-facing behavior. Record root cause and prevention.

INTERVIEW PHRASING	“I would not restart blindly. First I would identify the affected layer, collect metrics and logs, correlate with the last change, test a likely cause, apply a controlled fix, and verify recovery.”

Start with these five checks

```
date
hostname
uptime
whoami
sudo systemctl --failed
```

What they answer: time and host identity, load averages, the current account, and failed systemd units. If sudo is unavailable, run the checks allowed to your account and explain the permission limitation.

Safety rules

- Do not run `kill -9`, `docker rm -f`, `kubectl delete`, `rm -rf`, or broad permission changes as a first step.
- Before restarting, capture status and recent logs. For stateful workloads, assess replicas, disruption budgets, persistence, and recovery behavior.
- Do not paste credentials, access keys, tokens, customer data, or private logs into public tools or commits.
- In a real production incident, follow the team’s change, access, and escalation procedures.

2. Linux troubleshooting field notes

2.1 Command map

Question	Start here	What to inspect
Which process uses CPU?	`top`; `ps -eo pid,ppid,comm,%cpu,%mem --sort=-%cpu | head`	Top process, trend, owner, and command.
Is memory under pressure?	`free -h`; `vmstat 1 5`; `ps ... --sort=-%mem`	Available memory, swap activity, process memory.
Which filesystem is full?	`df -hT`; `df -ih`	Block space vs inode exhaustion.

Which directories are large?	`sudo du -xhd1 /var 2>/dev/null | sort -h`	Large top-level directories; filesystem boundary limited by `-x`.
Which process listens on a port?	`sudo ss -ltnp`	Listen address, port, owning process.
Is a service healthy?	`systemctl status SERVICE_NAME`; `journalctl -u SERVICE_NAME -n 100 --no-pager`	Unit state and recent service logs.
What errors did the kernel log?	`sudo journalctl -k -p warning --since "30 min ago"`	Kernel warnings, OOM, driver or I/O issues.
Can the host reach a service?	`curl -v --max-time 5 http://127.0.0.1:PORT/health`	DNS/connection/TLS/HTTP response; use correct URL.
What changed recently?	`journalctl --since "1 hour ago"`; deployment logs; `last -x | head`	Correlate incidents with recent changes.
What is mounted?	`findmnt`; `lsblk -f`	Filesystem/mount and block-device context.

2.2 High CPU: investigate, do not guess

```
uptime
top -o %CPU
ps -eo pid,ppid,user,stat,%cpu,%mem,etime,comm --sort=-%cpu | head -15
pidstat -u -p ALL 1 5
vmstat 1 5
```

Interpretation: `top` is a live snapshot; `ps` ranks current process usage; `pidstat` samples per-process CPU over time when installed; `vmstat` helps show run queue, memory, swap, and I/O activity. High load average alone does not prove CPU saturation. Check whether CPU is busy, tasks are blocked on I/O, or only one application thread is overloaded.

Scenario: process is at 95% CPU

19. Identify PID and owner using `ps` / `top`. Confirm whether it is the application or an expected batch job.
20. Sample over time. Check application logs, request volume, recent release, thread count, and per-process CPU trend.
21. If safe, inspect the process (`ps -fp PID`, `top -H -p PID` on Linux) and application-specific metrics.
22. Mitigate only with approval: reduce traffic, scale replicas, roll back a correlated bad release, or restart only when evidence supports it.
23. Verify CPU, latency, errors, and capacity return to a healthy range. Add an alert or regression test if appropriate.

AVOID	Do not blindly kill the process. It may be the database, a critical service, or doing important work. Capture evidence before acting.

2.3 Memory high / process may be leaking

```
free -h
vmstat 1 5
ps -eo pid,ppid,user,%mem,rss,vsz,comm --sort=-%mem | head -15
cat /proc/meminfo | head -20
dmesg -T 2>/dev/null | grep -i -E "out of memory|oom|killed process" | tail -20
```

`RSS` is resident memory currently in RAM; `VSZ` is virtual address space and is not the same as RAM consumed. Linux uses otherwise-free RAM for cache, so low “free” memory alone is not always an incident. Look at `available`, swap-in/out activity, growth over time, OOM messages, and container memory limits.

2.4 Disk full / log growth

```
df -hT
df -ih
sudo du -xhd1 /var 2>/dev/null | sort -h
sudo find /var/log -type f -size +100M -printf "%s %p\n" 2>/dev/null | sort -n | tail -20
sudo journalctl --disk-usage
sudo lsof +L1 2>/dev/null | head -30
```

`df -h` measures filesystem blocks; `df -i` measures inodes. `du` finds visible directory usage. A deleted file can still consume disk if a process has it open; `lsof +L1` can reveal deleted-but-open files. Use log rotation, application cleanup, retention policies, or volume expansion after identifying the actual cause. Do not delete active logs or unknown data just to free space.

2.5 Service down / restart loop

```
systemctl status nginx --no-pager
systemctl is-enabled nginx
sudo journalctl -u nginx --since "30 min ago" --no-pager
sudo nginx -t
sudo ss -ltnp | grep -E ":80|:443"
curl -v --max-time 5 http://127.0.0.1/
```

Replace `nginx` with the unit name. Check configuration validation before restarting. Common causes include invalid configuration, port already bound, missing file/permission, environment variable or secret missing, dependency failure, disk full, and process crash. A service being “active” does not prove the application endpoint is healthy.

2.6 Network / port unreachable

```
ip addr
ip route
getent hosts example.com
sudo ss -ltnp
curl -v --connect-timeout 3 --max-time 8 https://example.com/
# If installed, test a specific TCP port
nc -vz HOSTNAME 443
```

Debug in layers: DNS resolution, route, local listener, host firewall, cloud security group/network ACL, remote firewall, TLS, and application response. A successful ping is not a requirement for an HTTPS service to be reachable; ICMP may be blocked.

2.7 Logs and useful filters

```
journalctl -u nginx -n 100 --no-pager
journalctl -u nginx --since "today" --grep="error" --no-pager
sudo tail -n 100 /var/log/nginx/error.log
sudo grep -iE "error|failed|timeout|refused" /var/log/nginx/error.log | tail -50
# Follow new log entries
sudo tail -f /var/log/nginx/error.log
```

Use `grep -i` for case-insensitive search, `-E` for alternatives, `tail` for recent lines, and `-f` to follow appends. Verify that the log file path is correct for the distribution and service.

2.8 Process states, signals, and permissions

```
ps -ef | head
ps -o pid,ppid,stat,etime,cmd -p PID
kill -TERM PID
# Only after understanding the process and failed graceful shutdown:
kill -KILL PID
stat FILE
namei -l /path/to/file
chmod 640 FILE
chown SERVICE_USER:SERVICE_GROUP FILE
```

`TERM` asks a process to stop cleanly; `KILL` cannot be caught and should be a last resort. `chmod 640` means owner read/write, group read, others no access. `namei -l` reveals permissions on every directory component. Fix ownership and permissions to the least privilege required; avoid `chmod -R 777`.

3. Bash scripting for operations

3.1 Basic structure

```
#!/usr/bin/env bash
set -Eeuo pipefail

if [[ $# -lt 1 ]]; then
  printf "Usage: %s SERVICE_NAME\n" "$0" >&2
  exit 2
fi

service_name="$1"
if systemctl is-active --quiet "$service_name"; then
  printf "%s is active\n" "$service_name"
else
  printf "%s is not active\n" "$service_name" >&2
  systemctl status "$service_name" --no-pager || true
  exit 1
fi
```

Save as `check_service.sh`; run `chmod +x check_service.sh` then `./check_service.sh nginx`. `set -e` exits on many unhandled errors, `-u` catches unset variables, and `pipefail` propagates failures within a pipeline. These options have context-specific behavior, so handle expected failures explicitly with `if`, `||`, or status checks.

3.2 Variables, arguments, conditions, loops

```
name="nginx"
echo "$name"
echo "script=$0 first_arg=${1:-not-provided}"

if [[ -f /etc/os-release ]]; then
  cat /etc/os-release
fi

for path in /var/log/*.log; do
  [[ -e "$path" ]] || continue
  printf "%s %s bytes\n" "$(stat -c %s "$path")" "$path"
done

if [[ "$EUID" -ne 0 ]]; then
  echo "Run with appropriate privileges if required" >&2
fi
```

Quote variables as `"$name"` to avoid word splitting and wildcard expansion. Use `${1:-default}` when an argument may be absent. `[[ ... ]]` is Bash syntax; scripts using it should run with Bash, not `sh script.sh`.

3.3 Capture output and return codes

```
if output=$(systemctl is-active nginx 2>&1); then
  printf "status=%s\n" "$output"
else
  rc=$?
  printf "service check failed (exit=%s): %s\n" "$rc" "$output" >&2
fi

# Validate syntax without executing the script
bash -n check_service.sh
# Trace a script while debugging
bash -x check_service.sh nginx
```

The exit code of the previous command is in `$?`, so save it immediately. `bash -n` parses syntax without running commands. `bash -x` prints expanded commands and can expose secrets, so do not use it indiscriminately in secret-bearing scripts.

3.4 Script: collect a basic host health report

```
#!/usr/bin/env bash
set -u -o pipefail

report="${1:-host-health-$(date +%Y%m%d-%H%M%S).txt}"
{
  echo "=== TIME / HOST ==="
  date -Is
  hostname
  uptime
  echo "=== MEMORY ==="
  free -h
  echo "=== FILESYSTEMS ==="
  df -hT
  echo "=== TOP CPU PROCESSES ==="
  ps -eo pid,user,%cpu,%mem,comm --sort=-%cpu | head -10
  echo "=== FAILED UNITS ==="
  systemctl --failed --no-pager || true
} > "$report"

printf "Report saved to %s\n" "$report"
```

Run: `bash host_health.sh` or `bash host_health.sh /tmp/health.txt`. This creates a snapshot, not a diagnosis. Review the output and compare to a known healthy baseline.

3.5 Script: alert when a filesystem exceeds a threshold

```
#!/usr/bin/env bash
set -u -o pipefail

threshold="${1:-80}"
if ! [[ "$threshold" =~ ^[0-9]+$ ]] || (( threshold < 1 || threshold > 100 )); then
  echo "Usage: $0 [threshold_percent_1_to_100]" >&2
  exit 2
fi

df -P -x tmpfs -x devtmpfs | awk -v limit="$threshold" '
NR > 1 {
  usage=$5
  gsub(/%/, "", usage)
  if (usage + 0 >= limit) {
    printf "WARNING: %s is %s%% used (mount: %s)\n", $1, usage, $6
    found=1
  }
}
END { if (!found) print "No filesystem crossed the configured threshold" }'
```

Run: `bash disk_threshold.sh 80`. It ignores tmpfs/devtmpfs and reports filesystems at or above the threshold. Adapt exclusions and mount paths to the system. It reports only; it does not delete files.

3.6 Script: find common error lines in a log

```
#!/usr/bin/env bash
set -u -o pipefail

file="${1:-}"
if [[ -z "$file" || ! -r "$file" ]]; then
  echo "Usage: $0 READABLE_LOG_FILE" >&2
  exit 2
fi

grep -iE "error|failed|exception|timeout|connection refused|out of memory" "$file" | tail -100 || true
```

Run: `bash find_errors.sh /var/log/nginx/error.log`. A final `|| true` means “no matching lines is not a script failure”. Remove it or handle the `grep` exit code explicitly if a CI pipeline should fail when matches are absent.

3.7 Bash interview traps

Trap	Safer approach
Unquoted variable: `rm $file`	Use `rm -- "$file"` only after validating exact target and intent.
Parsing `ps` output for process names	Prefer `pgrep`, `pidof`, service managers, or structured process tools when possible.
Assuming `grep` returns 0 if no match	`grep` returns 1 when no lines match and >1 for errors; handle that intentionally.
`for x in $(cat file)`	Use `while IFS= read -r line; do ...; done < file` for line-oriented text.
`set -e` assumed to catch every failure	Check important commands explicitly and understand conditional/pipeline behavior.
No input validation	Validate argument count, numeric ranges, file readability, and allowed targets.

4. Python scripting for operations

4.1 Run a script

```
python3 --version
python3 script.py
python3 -m venv .venv
# Linux/macOS
source .venv/bin/activate
python -m pip install PACKAGE_NAME
```

Prefer the Python standard library for interview scripts. Install dependencies only when required. Use a virtual environment for project-specific packages. On Windows, venv activation is `.venv\Scripts\activate`.

4.2 Minimal structure, arguments, and safe subprocesses

```
import argparse
import subprocess
import sys

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("service", help="systemd service name")
    args = parser.parse_args()

    result = subprocess.run(
        ["systemctl", "is-active", args.service],
        text=True,
        capture_output=True,
        check=False,
        timeout=10,
    )

    status = result.stdout.strip() or result.stderr.strip()
    print(f"{args.service}: {status}")
    return 0 if result.returncode == 0 else 1

if __name__ == "__main__":
    sys.exit(main())
```

Save as `check_service.py`; run `python3 check_service.py nginx`. `subprocess.run` executes an external command. A list of arguments avoids shell parsing, `capture_output=True` collects stdout/stderr, `timeout` bounds the wait, and `returncode` indicates success/failure. `systemctl` is Linux-specific and might not be available in a container or Windows.

4.3 Read a log and print matching lines

```
from pathlib import Path
import argparse

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("logfile", type=Path)
    parser.add_argument("--last", type=int, default=200)
    args = parser.parse_args()

    if args.last < 1:
        parser.error("--last must be greater than zero")
    if not args.logfile.is_file():
        parser.error(f"not a file: {args.logfile}")

    keywords = ("error", "failed", "exception", "timeout", "refused")
    lines = args.logfile.read_text(errors="replace").splitlines()[-args.last:]
    matches = [line for line in lines if any(word in line.lower() for word in keywords)]

    print(f"Scanned {len(lines)} lines; matches={len(matches)}")
    for line in matches:
        print(line)

if __name__ == "__main__":
    main()
```

Save as `scan_logs.py`; run `python3 scan_logs.py /var/log/syslog --last 500`. Some logs require elevated permissions. For very large files, stream line-by-line rather than loading the entire file into memory; see the next example.

4.4 Stream a large log without loading it all

```
from pathlib import Path
import argparse
from collections import deque

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("logfile", type=Path)
    parser.add_argument("--last", type=int, default=500)
    args = parser.parse_args()
    if args.last < 1:
        parser.error("--last must be greater than zero")

    try:
        with args.logfile.open("r", errors="replace") as handle:
            recent = deque(handle, maxlen=args.last)
    except OSError as exc:
        parser.error(str(exc))

    for line in recent:
        if any(word in line.lower() for word in ("error", "failed", "timeout")):
            print(line, end="")

if __name__ == "__main__":
    main()
```

A bounded `deque` keeps only the last N lines while reading the file. For logs that rotate while being read or compressed logs, use a more specialized log pipeline.

4.5 Script: summarize CPU/memory snapshot

```
import platform
import shutil
import subprocess
from datetime import datetime

def run(command):
    return subprocess.run(command, text=True, capture_output=True, check=False, timeout=10)

def main():
    print("time:", datetime.now().astimezone().isoformat())
    print("host:", platform.node())
    print("os:", platform.platform())

    for command in (["uptime"], ["free", "-h"], ["df", "-hT"]):
        if shutil.which(command[0]):
            result = run(command)
            print(f"\n$ {.join(command)}\n{result.stdout}")
            if result.returncode != 0:
                print(result.stderr)

    if shutil.which("ps"):
        result = run(["ps", "-eo", "pid,user,%cpu,%mem,comm", "--sort=-%cpu"])
        print("\nTop CPU processes:")
        print("\n".join(result.stdout.splitlines()[:11]))

if __name__ == "__main__":
    main()
```

Save as `host_snapshot.py`; run `python3 host_snapshot.py`. It uses Linux-flavored command options; on non-Linux systems, command flags differ. This is a simple snapshot, not a cross-platform metrics agent.

4.6 Script: HTTP health check with timeout

```
from urllib.request import urlopen
from urllib.error import URLError, HTTPError
import argparse
import sys

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("url", help="health-check URL")
    parser.add_argument("--timeout", type=float, default=5)
    args = parser.parse_args()

    try:
        with urlopen(args.url, timeout=args.timeout) as response:
            print(f"HTTP {response.status} from {args.url}")
            return 0 if 200 <= response.status < 400 else 1
    except HTTPError as exc:
        print(f"HTTP error: {exc.code}", file=sys.stderr)
    except (URLError, TimeoutError, ValueError) as exc:
        print(f"Health check failed: {exc}", file=sys.stderr)
    return 1

if __name__ == "__main__":
    sys.exit(main())
```

Run: `python3 health_check.py http://127.0.0.1:8080/health`. An HTTP 200 from one URL is only one health signal; confirm that the URL checks the critical dependencies you care about. Avoid using this simplistic script as a production-grade monitor without retry, TLS, and alerting design.

4.7 Script: alert when files exceed a size threshold

```
from pathlib import Path
import argparse

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("directory", type=Path)
    parser.add_argument("--mb", type=float, default=100)
    args = parser.parse_args()

    if not args.directory.is_dir():
        parser.error(f"not a directory: {args.directory}")
    if args.mb <= 0:
        parser.error("--mb must be positive")

    limit = args.mb * 1024 * 1024
    matches = []
    for path in args.directory.rglob("*"):
        try:
            if path.is_file() and path.stat().st_size >= limit:
                matches.append((path.stat().st_size, path))
        except OSError:
            continue

    for size, path in sorted(matches, reverse=True):
        print(f"{size / 1024 / 1024:.1f} MB\t{path}")
    print(f"Files at least {args.mb:g} MB: {len(matches)}")

if __name__ == "__main__":
    main()
```

Run: `python3 large_files.py /var/log --mb 100`. Directory traversal may take time, and access restrictions are expected. It flags files only; it never deletes them.

4.8 Python habits interviewers like

- Use `argparse` instead of hard-coded paths and values.
- Use `pathlib` for paths and `with open(...)` for file handling.
- Use `subprocess.run([...], timeout=..., check=False)` and inspect return code and stderr. Avoid `shell=True` for untrusted input.
- Catch specific exceptions and print useful messages to stderr; do not silently swallow all errors.
- Use `logging` rather than scattered `print()` calls in a longer-running application.
- Return meaningful exit codes: 0 means success; nonzero means a failure or invalid input.
- Test empty files, missing files, permission errors, malformed inputs, timeouts, and unexpected command output.

5. Docker troubleshooting runbook

5.1 Fast triage commands

```
docker ps
docker ps -a
docker images
docker stats --no-stream
docker logs --tail 100 CONTAINER_NAME
docker inspect CONTAINER_NAME
docker port CONTAINER_NAME
docker events --since 30m
```

Symptom	Checks	Likely causes / next step
Container exits	`docker ps -a`; `docker logs NAME`; `docker inspect NAME`	App process ended, bad command, missing config, dependency error, OOM.
Restart loop	`docker inspect -f "{{.State.RestartCount}} {{.State.ExitCode}}" NAME`; logs	Entrypoint crash, config issue, health check, missing secret, OOM.
App inaccessible	`docker port NAME`; `docker inspect`; `curl` on host	Port not published, wrong bind address, wrong port, host firewall, app listens only on loopback inside container.
High memory/CPU	`docker stats`; host `free`, `top`; inspect limits	Memory leak, workload spike, CPU quota, container limit too low.
Cannot reach another container	`docker network ls`; `docker network inspect NETWORK`; `docker exec NAME getent hosts SERVICE`	Different networks, wrong DNS/service name, app not listening, network policy/firewall outside Docker.
Disk usage growing	`docker system df`; inspect volumes/logs	Unused images, writable layers, volumes, unbounded logs.

5.2 Common commands and meanings

```
docker run -d --name web -p 8080:80 nginx
docker logs -f --tail 100 web
docker exec -it web sh
docker inspect web
docker stats web
docker restart web
docker stop web
docker start web
docker compose ps
docker compose logs --tail 100
docker compose config
docker compose up -d --build
```

`-p 8080:80` maps host port 8080 to container port 80. `docker exec` runs a command in a running container. `docker compose config` validates/expands Compose configuration before rollout. Prefer `sh` because minimal images may not contain Bash.

5.3 Case: container is running but the site is down

24. Check process state and port publishing: `docker ps` and `docker port CONTAINER`.
25. Read recent application output: `docker logs --tail 100 CONTAINER`.
26. Probe from the host: `curl -v --max-time 5 http://127.0.0.1:HOST_PORT/health`.
27. Inspect environment, mounts, networks, and health status: `docker inspect CONTAINER`.
28. If app listens only on `127.0.0.1` inside the container, configure it to listen on the container interface (often `0.0.0.0`) when appropriate.
29. After an evidence-based fix, recreate/restart in a controlled way, then probe again and inspect logs.

5.4 Image/container/volume distinctions

Object	Purpose	Common caution
Image	Immutable template/layers used to create containers	A tag may move; pin specific versions/digests for repeatable deployments.
Container	Runtime instance with writable layer and process state	Removing it can destroy unpersisted writable-layer data.
Volume	Persistent data managed by Docker	`docker compose down -v` can delete declared named volumes; do not use casually.
Network	Connectivity and name resolution between containers	Containers on different user-defined networks may not resolve each other by service name.
Health check	Tests application condition inside container	A running container may still be unhealthy; understand the test and failure threshold.

5.5 Cleanup safely

```
docker system df
docker image ls
docker container ls -a
docker volume ls
# Preview targets and confirm owners before removing specific unused objects
docker image prune
docker container prune
```

CAUTION Prune commands delete resources. Review what will be removed, confirm nothing is needed for rollback, and never delete volumes without understanding the data lifecycle.

6. Kubernetes troubleshooting runbook

6.1 First-pass commands

```
kubectl get nodes -o wide
kubectl get pods -A -o wide
kubectl get deployments -A
kubectl get services -A
kubectl get events -A --sort-by=.lastTimestamp
kubectl get pods -n NAMESPACE
kubectl describe pod POD_NAME -n NAMESPACE
kubectl logs POD_NAME -n NAMESPACE --tail=100
kubectl logs POD_NAME -n NAMESPACE --previous --tail=100
kubectl rollout status deployment/DEPLOYMENT -n NAMESPACE
kubectl rollout history deployment/DEPLOYMENT -n NAMESPACE
```

Use `-n NAMESPACE` consistently. `-A` shows all namespaces and can be noisy. `--previous` requests logs from the prior terminated container instance when available. Events are time-sensitive and should be gathered early.

6.2 Pod states and what to check

State / symptom	Inspect	Typical cause / response
Pending	`kubectl describe pod`; node resources; events	No schedulable node, insufficient CPU/memory, unbound PVC, taint/affinity mismatch.
CrashLoopBackOff	`kubectl logs --previous`; `describe pod`; exit code	Application crash, invalid config, missing secret, failing dependency, wrong command.
ImagePullBackOff / ErrImagePull	Pod events; image name/tag; registry credentials	Bad tag, missing image, registry auth/network issue, image pull secret.
OOMKilled	Pod last state; requests/limits; app memory trend	Container exceeded memory limit or node memory pressure; profile/fix or right-size limit.
Running but not Ready	`describe pod`; readiness probe config and events	App has not initialized, wrong probe path/port, dependency unavailable.
Service has no endpoints	`kubectl get endpointslices`; service selector vs pod labels; readiness	Selector mismatch or pods not Ready; check labels and target port.
Node NotReady	`kubectl describe node`; events; kubelet/cloud health	Node pressure, kubelet/network/runtime issue, underlying instance issue.

6.3 CrashLoopBackOff workflow

```
kubectl get pod POD_NAME -n NAMESPACE -o wide
kubectl describe pod POD_NAME -n NAMESPACE
kubectl logs POD_NAME -n NAMESPACE --previous --tail=200
kubectl logs POD_NAME -n NAMESPACE --all-containers=true --tail=100
kubectl get pod POD_NAME -n NAMESPACE -o yaml
```

30. Find last termination reason, exit code, restart count, and recent events.
31. Read previous container logs; verify the entrypoint/command, required environment, ConfigMaps and Secrets, file mounts, and permissions.
32. Check liveness/readiness/startup probes. A too-aggressive liveness probe can repeatedly kill a slow-starting app.
33. Check resource requests/limits and dependency connectivity.
34. Apply a narrow manifest fix, observe rollout and pod events, then verify the application from the service path.

6.4 Service works inside pod but not from outside

```
kubectl get svc SERVICE_NAME -n NAMESPACE -o yaml
kubectl get endpointslices -n NAMESPACE -l kubernetes.io/service-name=SERVICE_NAME
kubectl get pods -n NAMESPACE --show-labels
kubectl describe ingress -n NAMESPACE
kubectl port-forward service/SERVICE_NAME 8080:80 -n NAMESPACE
```

Compare the Service selector to pod labels, `port` to `targetPort`, and readiness/endpoints. Then check ClusterIP vs LoadBalancer/NodePort, ingress rules, DNS, cloud load balancer, security groups, network policy, and TLS. Port-forward is a diagnostic bypass and does not prove external routing is configured correctly.

6.5 Rollout and rollback

```
kubectl rollout status deployment/DEPLOYMENT -n NAMESPACE
kubectl rollout history deployment/DEPLOYMENT -n NAMESPACE
kubectl describe deployment DEPLOYMENT -n NAMESPACE
# Roll back only after confirming the bad rollout and intended revision
kubectl rollout undo deployment/DEPLOYMENT -n NAMESPACE
kubectl rollout status deployment/DEPLOYMENT -n NAMESPACE
```

CAUTION Rollback changes live workloads. Confirm revision, impact, database/schema compatibility, and team authorization first. A manifest rollback may not undo an irreversible database migration.

6.6 Requests, limits, probes, and replica count

Setting	Meaning	Failure mode if wrong
CPU request	Scheduling reservation used for placement	Too high: pods unschedulable; too low: contention/poor placement.
CPU limit	CPU ceiling enforced by the runtime	Too low can throttle app and cause latency without OOM.
Memory request	Memory scheduling reservation	Too high: unschedulable; too low: crowded node placement.
Memory limit	Container memory ceiling	Crossing it can lead to OOMKilled.
Readiness probe	Whether pod should receive service traffic	Bad probe can remove healthy pods from endpoints.
Liveness probe	Whether container should be restarted	Bad/aggressive probe can create restart loops.

Startup probe	Allows slow startup before liveness/readiness checks take over	Missing it can kill slow-starting applications.

6.7 Useful but potentially disruptive commands

```
# Scale a deployment (state-changing)
kubectl scale deployment/DEPLOYMENT --replicas=3 -n NAMESPACE

# Delete one pod only when the controller and impact are understood
kubectl delete pod POD_NAME -n NAMESPACE

# Apply manifests after reviewing the diff / change process
kubectl diff -f k8s/
kubectl apply -f k8s/
```

Scaling is not automatically a fix: downstream capacity, quotas, database connections, and costs may be limiting. Deleting a pod managed by a controller typically causes it to be recreated, but that can still interrupt traffic or worsen an incident.

7. AWS / EC2 / CloudWatch operational checks

7.1 Check account, region, and identity first

```
aws --version
aws sts get-caller-identity
aws configure list
aws ec2 describe-instances --region ap-south-1 --instance-ids INSTANCE_ID --query "Reservations[].Instances[].
{Id:InstanceId,State:State.Name,Type:InstanceType,AZ:Placement.AvailabilityZone,PublicIP:PublicIpAddress}" --output table
```

Always verify the account/role and region before actions. `get-caller-identity` shows which identity the CLI is using. Do not run mutating AWS commands from a context you have not verified.

7.2 EC2 instance is unreachable

35. Confirm the instance is running and the correct region/account/instance ID is selected.
36. Check status checks, instance/system reachability, recent system log or console screenshot where applicable.
37. Validate the route table, subnet, internet gateway/NAT design, public IP, security group, network ACL, and local firewall.
38. Confirm SSH/RDP service is listening and the username/key are correct. Restrict inbound access to the required source instead of opening to everyone.
39. Use Session Manager if configured and authorized; confirm its IAM permissions and SSM agent/network prerequisites.
40. For urgent recovery, preserve evidence and follow the organization’s approved recovery path; do not terminate/recreate blindly if data or state matters.

7.3 EC2 CPU metric using CloudWatch

```
aws cloudwatch get-metric-statistics \
  --namespace AWS/EC2 \
  --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value=INSTANCE_ID \
  --start-time 2026-10-25T11:00:00Z \
  --end-time 2026-10-25T12:00:00Z \
  --period 300 \
  --statistics Average Maximum \
  --region ap-south-1 \
  --output table
```

Replace the time window with the incident period in UTC and use the actual region/instance ID. CloudWatch returns metric points that may not be chronologically ordered. Standard EC2 monitoring includes CPU utilization; guest memory and filesystem metrics normally require an agent or other telemetry.

7.4 Instance inventory and security checks

```
aws ec2 describe-instances --region ap-south-1 \
  --query "Reservations[].Instances[].
{Id:InstanceId,State:State.Name,Type:InstanceType,PrivateIP:PrivateIpAddress,PublicIP:PublicIpAddress}" \
  --output table

aws ec2 describe-security-groups --region ap-south-1 \
  --group-ids sg-XXXXXXXX \
  --output json
```

The examples are read-only. For access failures, inspect which security group is attached to the instance, source CIDR, target port, subnet route, network ACL, and host firewall. Avoid broad ingress such as SSH from `0.0.0.0/0`; use least privilege and approved access patterns.

7.5 IAM AccessDenied workflow

- Check the exact AWS identity and role with `aws sts get-caller-identity`.
- Record the failing API action, resource ARN, region, and full error message.
- Review identity policies, resource-based policies, permission boundaries, session policies, SCPs, and explicit denies.
- Check whether the required resource policy or trust policy is missing, and whether the assumed role can be assumed from that principal.
- Request the minimum required permission from the account owner. Do not “fix” it by attaching AdministratorAccess.

7.6 AWS CLI safety reminders

```
aws configure list
aws sts get-caller-identity
aws ec2 describe-instances --region ap-south-1 --output table
# Use dry-run when the specific command supports it
aws ec2 run-instances help
```

`aws configure list` can show credential source and region metadata; never copy secrets from terminal output into documentation. Not every AWS command supports `--dry-run`. Review the exact API semantics before changing, deleting, or recreating cloud resources.

8. CI/CD deployment failure runbook

Your repository already has GitHub Actions notes. This section focuses on the debug logic behind a broken pipeline, not a duplicate workflow tutorial.

8.1 Read the pipeline from the failing boundary

41. Identify the first failed step, not just the final red job. Read the complete error and the lines above it.
42. Confirm trigger, branch, commit SHA, workflow file, runner OS, working directory, environment variables, and dependency versions.
43. For build/test failures, reproduce the same command locally or in a disposable container where feasible.
44. For credential failures, verify secret/role name, scope, expiry, trust relationship, and permissions. Never print secret values.
45. For Docker push failures, check registry login, repository name, tag, permissions, and image architecture/size.
46. For deploy failures, verify the target environment, cluster/context/namespace, manifest, image tag/digest, rollout events, and readiness.
47. Re-run only after fixing the root cause or confirming a transient external failure. Verify the deployed version and health after a green pipeline.

8.2 Typical failure mapping

Error	Likely explanation	What to check
Command not found	Tool missing or PATH setup incorrect	Runner image, setup action, version and PATH.
Permission denied	File not executable or identity lacks access	`ls -l`, runner user, IAM/RBAC, ownership and mode.
Tests pass locally, fail in CI	Environment/config/dependency mismatch	Versions, case-sensitive paths, env vars, time zone, services, test order.
Secret empty / access denied	Secret scope, role assumption, trust or permission issue	Repository/environment secret scope, OIDC subject/audience, IAM action/resource.
Image not found after build	Wrong tag/registry/repository or build/push job did not run	Job dependency, image tag, registry and artifact outputs.
Kubernetes rollout timeout	New pods not Ready or old pods cannot drain	`kubectl describe`, logs, events, probe config, image pull, resource capacity.
Terraform drift/plan mismatch	Real infrastructure differs from state/config	Review plan, state backend, workspace/account, and out-of-band changes.

8.3 Deployment health verification checklist

- Correct commit SHA, image tag, or immutable digest was deployed.
- Desired replicas are available and rollout completed.
- Health/readiness endpoint returns expected status and dependencies work.
- Error rate, response latency, CPU, memory, restarts, and logs look normal.
- Rollback path and previous artifact/image are known before declaring deployment complete.

9. High availability and performance scenarios

9.1 Availability is a system property

High availability is not simply “run two instances”. Consider replicas across failure domains, load balancing and health checks, autoscaling, storage and database availability, dependency timeouts, backups/restore, deployment strategy, observability, and tested recovery. Redundancy in one layer does not eliminate a single point of failure in another.

Layer	Questions to ask
Compute	Can one instance/node fail without taking the service down? Are there enough healthy replicas?
Traffic	Does the load balancer route only to healthy targets? Are timeout and connection settings sensible?
Application	Does it handle retries, timeouts, idempotency, and graceful shutdown?
Data	Are backups restorable? Is replication consistent? What is the RPO/RTO?
Deployment	Can the release roll out gradually? Is rollback safe with schema changes?
Observability	Do alerts detect symptoms users feel, not only CPU? Are logs, metrics and traces correlated?

9.2 High latency, CPU looks normal

Look for database queries, connection pools, lock contention, disk I/O, network latency, DNS, thread/worker saturation, external APIs, and queue backlog. Check p95/p99 latency and request/error counts, not only averages. “CPU is low” does not prove the host is healthy.

9.3 App returns HTTP 502 / 503 / 504

Code	Common meaning (depends on proxy/application)	Initial checks
502 Bad Gateway	Proxy received invalid response from upstream	Upstream target health, app port/listener, protocol mismatch, app crash, proxy logs.
503 Service Unavailable	Service unable to handle request / no healthy backends	Healthy target count, pods/endpoints, capacity, maintenance, rate limiting.
504 Gateway Timeout	Upstream did not respond before timeout	Slow dependency, blocked worker pool, network latency, too-short timeout, saturated DB.

9.4 OOMKilled in Kubernetes but node has memory

A container can hit its own memory limit while the node still has available memory. Compare container RSS/working set to its configured limit, inspect `lastState.terminated.reason`, profile memory growth, and determine whether the

limit or application behavior is wrong. Raising a limit without checking node capacity may move the failure to the node.

9.5 Deployment works in test but fails in production

Compare immutable artifact/image, configuration, secrets, service dependencies, IAM/RBAC, network rules, DNS/TLS, schema version, resource limits, and traffic. Prefer configuration diff and targeted health checks over rebuilding a “similar” artifact for production.

10. Mock interview scenarios

Practice saying what you would inspect first and why. A senior engineer values disciplined debugging, clear communication, and safe changes more than a memorized list of commands.

Scenario 1: Linux CPU is 95%.

Use `top` and sorted `ps`; sample over time using `pidstat` if available; compare load/run queue and I/O; identify process and correlate with traffic/release/logs; mitigate safely; verify latency and CPU.

Scenario 2: Disk is 100% full.

Run `df -hT` and `df -ih`; find largest directories using `du`; inspect large logs and deleted-open files with `lsof +L1`; preserve data; fix retention/log rotation or expand volume; verify free space and app recovery.

Scenario 3: Service says active but site cannot load.

Check `systemctl status`, service logs, process listener with `ss`, local `curl`, config test, route/firewall and upstream/dependency. A running process is not proof the endpoint works.

Scenario 4: Write a Bash CPU alert.

Use `ps`/`top` for inspection or `/proc` with sampling for accurate interval utilization. Validate thresholds as numbers; print warning and exit nonzero; avoid killing processes automatically.

Scenario 5: Python job must monitor health endpoint.

Use `urllib` or `requests` with timeout; validate expected status; catch timeout/HTTP errors; log timestamp and result; return nonzero on failure; schedule via cron/systemd/GitHub Actions appropriate to purpose.

Scenario 6: Docker container exits immediately.

Check `docker ps -a`, logs, exit code/state from inspect, entrypoint/CMD, environment/config, and whether the app stays in foreground. Fix the actual startup failure.

Scenario 7: Kubernetes pod is CrashLoopBackOff.

Describe pod, inspect events and previous logs, termination exit code/reason, probes, config/secrets, mounts, dependency access, and resource limits.

Scenario 8: Kubernetes service has no endpoints.

Check service selector and pod labels, readiness, endpointslices, port/targetPort, and whether pods belong to expected namespace.

Scenario 9: A GitHub Actions pipeline is failing.

Find first failed step; inspect runner/tool versions, checkout, commands, artifacts, secret/role scope, and deploy target. Reproduce and fix before rerun; verify live rollout.

Scenario 10: EC2 is not reachable over SSH.

Check account/region/instance state and status checks, public/private network path, route and gateway, security group/NACL, host firewall and SSH service, user/key; use approved Session Manager if available.

Scenario 11: New deploy causes 5xx errors.

Check change correlation, target/pod health, logs, rollout, image/config, dependency compatibility, and metrics. If evidence supports regression, follow approved rollback path and verify recovery.

Scenario 12: Need high availability for a small web app.

Multiple replicas across failure domains, load balancer health checks, autoscaling/capacity, resilient data/dependencies, monitoring/alerts, backups/restore, safe deployment and tested rollback.

Scenario 13: Python script hangs.

Add bounded timeouts to subprocess/network calls, inspect blocking operation, use `capture_output` and return codes, log context and handle TimeoutExpired. Do not leave unbounded production tasks.

Scenario 14: API response is slow but CPU/memory look normal.

Break down request timing; look at p95/p99, database and external call latency, connection pools, disk/network I/O, locks, queue backlog, DNS/TLS and traces.

Scenario 15: AccessDenied on AWS.

Confirm caller identity; capture action/resource/region and full error; check role trust, identity/resource policy, boundary/SCP/session policy and explicit deny; request least-privilege change.

Scenario 16: App was OOMKilled.

Confirm container/node state and limits; inspect memory trend and logs; distinguish limit exceedance from node pressure; profile leak or load; right-size only after capacity and root cause checks.

10.1 Answer structure to say out loud

1. Impact: Who/what is affected and since when?
2. Scope: Host, process, container, cluster, cloud, network, or dependency?
3. Evidence: Which command/metric/log will discriminate between causes?
4. Hypothesis: What does the evidence suggest, and what would disprove it?
5. Mitigation: What is the smallest safe/reversible action?
6. Verification: Which user-visible metric/health check proves recovery?
7. Prevention: What alert, test, limit, or runbook prevents recurrence?

11. Quick command index

Need	Commands
CPU	`top`; `ps -eo pid,%cpu,%mem,comm --sort=-%cpu | head`; `pidstat -u 1 5`
Memory	`free -h`; `vmstat 1 5`; `ps ... --sort=-%mem`; inspect OOM logs
Disk space	`df -hT`; `df -ih`; `du -xhd1 PATH`; `lsof +L1`
Logs	`journalctl -u UNIT -n 100 --no-pager`; `tail -f FILE`; `grep -iE`

Processes	`ps -ef`; `pgrep -a NAME`; `kill -TERM PID`
Ports/network	`ss -ltnp`; `ip addr`; `ip route`; `getent hosts NAME`; `curl -v`
Systemd	`systemctl status UNIT`; `systemctl is-active UNIT`; `journalctl -u UNIT`
Docker	`docker ps -a`; `docker logs`; `docker inspect`; `docker stats`; `docker port`
Kubernetes	`kubectl get`; `kubectl describe`; `kubectl logs --previous`; `kubectl get events`; `kubectl rollout status`
AWS	`aws sts get-caller-identity`; `aws configure list`; `aws ec2 describe-instances`; CloudWatch metrics
Bash syntax/debug	`bash -n script.sh`; `bash -x script.sh` (beware secrets)
Python	`python3 --version`; `python3 script.py --help`; check stdout/stderr/return code and timeout

Official documentation and further reading

Linux performance investigation

https://learn.microsoft.com/en-us/troubleshoot/azure/virtual-machines/linux/troubleshoot-performance-bottlenecks-linux

GNU Bash Reference Manual

https://www.gnu.org/s/bash/manual/bash.html

Python subprocess documentation

https://docs.python.org/3/library/subprocess.html

Python logging tutorial

https://docs.python.org/3/howto/logging.html

Docker container logs reference

https://docs.docker.com/reference/cli/docker/container/logs/

Kubernetes application troubleshooting

https://kubernetes.io/docs/tasks/debug/debug-application/

Kubernetes pod debugging

https://kubernetes.io/docs/tasks/debug/debug-application/debug-pods/

AWS CLI: EC2 describe-instances

https://docs.aws.amazon.com/cli/latest/reference/ec2/describe-instances.html

AWS CLI: CloudWatch get-metric-statistics

https://docs.aws.amazon.com/cli/latest/reference/cloudwatch/get-metric-statistics.html

Before the interview: 10-minute check

- Can you explain each command you run and what result would change your hypothesis?
- Can you distinguish a symptom (CPU high) from a cause (runaway loop, traffic surge, lock, or I/O wait)?

- Can you write a short Bash script with input validation and a clear exit code?
- Can you write a small Python script that calls a command/endpoint with timeout and handles failures?
- Can you troubleshoot CrashLoopBackOff, ImagePullBackOff, no service endpoints, and a failed rollout?
- Can you confirm AWS identity/region and explain the minimum networking checks for an unreachable EC2?
- Can you explain how to verify a deployment and when not to restart or delete anything?

FINAL PRINCIPLE	Do not bluff. State what you know, name the evidence you need, and explain how you would verify the fix. A safe, methodical diagnosis is more credible than an untested command.